

Week10:Interval Cover

Questions

1. Analyze worst case time complexity of above algorithm.

$O(n^2)$

2. Analyze worst case time complexity of above algorithm with sorted V .

$O(n)$

3. Rewrite above algorithm to detect target range may impossible to be covered by original interval set. Analyze space complexity, time complexity of best case and worst case of modified algorithm.

當找不到任何區間可以更新 p 且 $p < r^*$ ，代表目標範圍不可能被原始區間集合覆蓋。時間複雜度為 $O(n)$ ，空間複雜度為 $O(n)$ 。

4. Above algorithm is doing through left to right. Please rewrite it to a through right to left version. Is your new algorithm always get the same result of original one? Give example(s) to explain your answer.

將 p 初始化為 r^* ，並從右至左搜尋符合 $r_v \geq p$ 且具有最小 l_v 的區間。若找不到任何區間可以更新 p 且 $p > l^*$ ，則代表目標範圍不可能被原始區間集合覆蓋。由於是從右至左搜尋，所以組成線段可能不同。但最小集合的區間數量一定相同。

C++

```
#include<iostream>
#include<vector>
#include<algorithm>
#include<cmath>
#include<set>

using namespace std;

#define pdd pair<double, double>

int grace_cover(vector<pdd> &segs, double l)
{
    sort(segs.begin(), segs.end());
    double current_end = 0, farthest_end = 0;
    int num_sprinklers = 0, i = 0;
    int n = segs.size();

    while (current_end < l)
    {
        while (i < n && segs[i].first <= current_end)
        {
            farthest_end = max(farthest_end, segs[i].second);
            i++;
        }
    }
}
```

```
    }
    if (current_end == farthest_end)
        return -1;
    current_end = farthest_end;
    num_sprinklers++;
}

return num_sprinklers;
}

int main()
{
    int n, l;
    uint32_t w, p, r;

    while (cin >> n >> l >> w)
    {
        set<pdd> unique_segs;
        for (int i = 0; i < n; i++)
        {
            cin >> p >> r;
            if (2 * r <= w)
                continue;
            double half_len = sqrt(r * r - (w * w / 4.0));
            unique_segs.emplace(p - half_len, p + half_len);
        }
        vector<pdd> segs(unique_segs.begin(), unique_segs.end());
        cout << grace_cover(segs, l) << endl;
    }
    return 0;
}
```